

GenSpark Forward Deployed Engineer 018-260818-FDE

Week 6: Build a RAG System from Scratch

Assignment submission, Mini RAG

Ken Mann · generated September 25, 2026 22:48 · repo: github.com/kenmann01/minirag (feat/rag)

Eleven deliverables in assignment order, plus a pipeline-trace appendix.

Each item is labeled: PROVIDED LIVE RENDERED STALE MISSING

Contents and evidence status

#	Deliverable	status
1	Source documents with the planted data quality issue	RENDERED
2	Minimal embed-store-retrieve loop (day-one proof)	LIVE
3	Chunking, embedding, and vector store code	RENDERED
4	Basic RAG pipeline end to end	LIVE
5	Hybrid retrieval code and where it beats vector-only	LIVE
6	Reranking code	RENDERED
7	Evaluation test set and harness code	RENDERED
8	Evaluation harness terminal output with recall and accuracy	LIVE
9	Planted-issue question, flawed answer, and written diagnosis	LIVE
10	Source attribution output	LIVE
11	Passing pipeline run (CI)	LIVE

Deliverable 1: Source documents with the planted data quality issue

All ten policy files with their ingest-parsed header blocks. The planted defect: **minion_expense_policy_2021.md** and **minion_expense_policy_2024.md** share Document ID **LAIR-POL-009** with conflicting numbers; the 2021 file carries Superseded-By: minion_expense_policy_2024.md, the lineage pointer the retriever filters on.

file	header block (as parsed at ingest)
minion_communications_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-019 **Effective Date:** February 10, 2024 **Last Reviewed:** February 10, 2024 **Owner:** Dr. Nefario (he monitors the channels; he says it is for safety; it is also for entertainment)
minion_dress_code_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-017 **Effective Date:** January 20, 2024 **Last Reviewed:** January 20, 2024 **Owner:** Gru (the final authority on capes, and there will be no capes)
minion_equipment_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-016 **Effective Date:** February 15, 2024 **Last Reviewed:** February 15, 2024 **Owner:** Dr. Nefario (custody of the clipboard is custody of the arsenal)
minion_expense_policy_2021.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-009 **Effective Date:** June 1, 2021 **Superseded-By:** minion_expense_policy_2024.md **Last Reviewed:** June 1, 2021 **Owner:** Dr. Nefario (younger, marginally less tired)
minion_expense_policy_2024.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-009 **Effective Date:** January 15, 2024 **Last Reviewed:** January 15, 2024 **Owner:** Dr. Nefario (Gru signs it, Nefario actually reads it)
minion_pto_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-011 **Effective Date:** January 1, 2024 **Last Reviewed:** January 1, 2024 **Owner:** Gru (with reluctant help from Dr. Nefario, who actually files the forms)
minion_remote_work_policy.md	**Organization:** Gru's Evil Lair & Associates (formerly Evil, now... light evil. Freelance evil.) **Document ID:** LAIR-POL-014 **Effective Date:** the day after Kevin flooded the sub-basement **Last Reviewed:** by Gru, personally, with much sighing **Owner:** Gru (nobody else can read the handwriting)
minion_training_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-018 **Effective Date:** February 1, 2024 **Last Reviewed:** February 1, 2024 **Owner:** Dr. Nefario (the slides are forty years old and still accurate)
minion_travel_security_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-015 **Effective Date:** March 1, 2024 **Last Reviewed:** March 1, 2024 **Owner:** Dr. Nefario (Gru reviews it from a helicopter, waving)
minion_visitors_policy.md	**Organization:** Gru's Evil Lair & Associates **Document ID:** LAIR-POL-020 **Effective Date:** March 15, 2024 **Last Reviewed:** March 15, 2024 **Owner:** Dr. Nefario (the sign-in sheet is older than most of the lair)

The seven conflicting rules (2021 vs 2024)

rule	2021 (superseded)	2024 (current)
receipt-free threshold	under \$20	under \$25
submission window	90 days	60 days
approval tier 1	under \$250	under \$500
approval tier 2	\$250-\$1,000	\$500-\$2,000
approval tier 3	over \$1,000	over \$2,000
mileage	\$0.56/mile	\$0.67/mile
per diem domestic / intl	\$60 / \$80	\$75 / \$95
entertainment cap	\$100/person	\$150/person

Deliverable 2: Minimal embed-store-retrieve loop (day-one proof)

Embeds two known texts, stores both, and proves the mileage line wins the cosine comparison for the query. Same embedder and store as the pipeline.

LIVE python minimal_loop.py

```
closest: Mileage in the Minion Van is reimbursed at $0.67 per mile.  
distance: 0.2613
```

Retrieved the mileage line, not the lockdown drill line: loop proven.

Deliverable 3: Chunking, embedding, and vector store code

Small-to-big chunking: child windows (800-1200 chars, sentence-aligned, 120+ overlap) are embedded and keyword-indexed; the full parent section rides along and is what the prompt sees.

CODE app/chunking.py

```
1  # Internal and Confidential - Not for External Distribution.
2  """Parent sections and child windows for policy retrieval.
3
4  Child windows target 800-1200 characters with 120-150 characters of overlap.
5  A flat 500 character window with 50 characters of overlap shatters a section's
6  rules into arbitrary pieces. One vector for a whole long section blurs those
7  rules together. Small-to-big chunking keeps a crisp child embedding for search
8  and the whole parent section for generation, because the answer prompt requires
9  every condition of the rule.
10 """
11
12 import re
13 from pathlib import Path
14
15 _HEADING = re.compile(r"^## (\d+)\. (.+)$", re.MULTILINE)
16 _EFFECTIVE_DATE = re.compile(r"^.*Effective Date:.*\s*(.+)$", re.MULTILINE)
17 _SUPERSEDED_BY = re.compile(r"^.*Superseded-By:.*\s*(.+)$", re.MULTILINE)
18 _SENTENCE_BREAK = re.compile(r"(?<=[!?\])\s+")
19
20 _WINDOW_MIN = 800
21 _WINDOW_TARGET = 1000
22 _WINDOW_MAX = 1200
23 _OVERLAP_MIN = 120
24
25
26 def _sentence_spans(body: str) -> list[tuple[int, int]]:
27     spans: list[tuple[int, int]] = []
28     start = 0
29     for match in _SENTENCE_BREAK.finditer(body):
30         spans.append((start, match.start()))
31         start = match.end()
32     if start < len(body):
33         spans.append((start, len(body)))
34     return [(span_start, span_end) for span_start, span_end in spans if body[span_start:span_end].strip()]
35
36
37 def _span_length(body: str, spans: list[tuple[int, int]], start: int, end: int) -> int:
38     if start >= end:
39         return 0
40     return len(body[spans[start][0] : spans[end - 1][1]].strip())
41
42
43 def _child_windows(body: str) -> list[str]:
44     """Grow sentence-span windows into 800-1200 character child chunks.
45
46     Windows stop growing at the target size and always overlap the next
47     window by at least ``_OVERLAP_MIN`` characters so a rule split across
48     the boundary stays retrievable from either side.
49     """
50     spans = _sentence_spans(body)
51     if not spans:
52         return []
53     if _span_length(body, spans, 0, len(spans)) <= _WINDOW_MAX:
54         return [body.strip()]
55
56     windows: list[str] = []
57     start = 0
58     while start < len(spans):
59         end = start + 1
60         while end < len(spans):
61             current = _span_length(body, spans, start, end)
62             proposed = _span_length(body, spans, start, end + 1)
63             if proposed > _WINDOW_MAX and current >= _WINDOW_MIN:
64                 break
65             if current >= _WINDOW_MIN and proposed > _WINDOW_TARGET:
66                 break
67             if proposed > _WINDOW_MAX:
68                 break
```

```

69         end += 1
70         windows.append(body[spans[start][0] : spans[end - 1][1]].strip())
71         if end >= len(spans):
72             break
73         next_start = end - 1
74         while next_start > start and _span_length(body, spans, next_start, end) < _OVERLAP_MIN:
75             next_start -= 1
76         if next_start <= start:
77             next_start = start + 1
78         start = next_start
79     return windows
80
81
82 def split(markdown: str, source_doc: str) -> list[dict]:
83     """Split one policy document into parent sections and child windows.
84
85     Args:
86         markdown: Complete policy text with numbered second-level headings.
87         source_doc: File name of the source policy, used in chunk identifiers.
88
89     Returns:
90         Chunk dictionaries holding section metadata, the full parent text,
91         and one child window per chunk.
92     """
93     effective = _EFFECTIVE_DATE.search(markdown)
94     superseded = _SUPERSEDED_BY.search(markdown)
95     effective_date = effective.group(1).strip() if effective else None
96     superseded_by = superseded.group(1).strip() if superseded else None
97     stem = Path(source_doc).stem
98
99     headings = list(_HEADING.finditer(markdown))
100     chunks: list[dict] = []
101     for index, heading in enumerate(headings):
102         body_start = heading.end()
103         body_end = headings[index + 1].start() if index + 1 < len(headings) else len(markdown)
104         section = heading.group(1)
105         body = markdown[body_start:body_end].strip()
106         for child_index, window in enumerate(_child_windows(body), start=1):
107             chunks.append(
108                 {
109                     "chunk_id": f"{stem}:s{section}:c{child_index:02d}",
110                     "source_doc": source_doc,
111                     "section": section,
112                     "section_title": heading.group(2).strip(),
113                     "effective_date": effective_date,
114                     "superseded_by": superseded_by,
115                     "parent_text": body,
116                     "text": window,
117                 }
118             )
119     return chunks

```

CODE app/embeddings.py

```

1  # Internal and Confidential - Not for External Distribution.
2  """Create normalized vector embeddings with the configured sentence-transformers model."""
3
4  from contextlib import contextmanager
5
6  from huggingface_hub import logging as hf_logging
7  from huggingface_hub.utils import (
8      are_progressBars_disabled,
9      disable_progressBars,
10     enable_progressBars,
11 )
12 from transformers.utils import logging as transformers_logging
13
14 from app.config import get_settings
15
16 _model = None
17 _loaded_name: str | None = None
18 _vectors: dict[tuple[str, str], list[float]] = {}
19
20
21 @contextmanager
22 def _quiet_model_loading():
23     """Suppress model-loading logs and progress bars within the context."""
24

```

```

25 hf_level = hf_logging.get_verbosity()
26 hf_progress_disabled = are_progressBars_disabled()
27 transformers_level = transformers_logging.get_verbosity()
28 transformers_progress_enabled = transformers_logging.is_progress_bar_enabled()
29 hf_logging.set_verbosity_error()
30 disable_progressBars()
31 transformers_logging.set_verbosity_error()
32 transformers_logging.disable_progress_bar()
33 try:
34     yield
35 finally:
36     hf_logging.set_verbosity(hf_level)
37     if not hf_progress_disabled:
38         enable_progressBars()
39     transformers_logging.set_verbosity(transformers_level)
40     if transformers_progress_enabled:
41         transformers_logging.enable_progress_bar()
42
43
44 def embed_texts(texts: list[str]) -> list[list[float]]:
45     """Embed text as normalized vectors with a lazily loaded shared model.
46
47     A text already encoded by the configured model in this process is returned
48     from memory. A different configured model loads separately and does not
49     reuse those vectors.
50
51     Args:
52         texts: Text values to encode in input order.
53
54     Returns:
55         One floating-point embedding per input text.
56     """
57     global _model, _loaded_name
58     model_name = get_settings().embedding_model
59     missing = [
60         text
61         for text in dict.fromkeys(texts)
62         if (model_name, text) not in _vectors
63     ]
64     if missing:
65         if _model is None or _loaded_name != model_name:
66             from sentence_transformers import SentenceTransformer
67
68             with _quiet_model_loading():
69                 _model = SentenceTransformer(model_name)
70                 _loaded_name = model_name
71             encoded = _model.encode(missing, normalize_embeddings=True)
72             for text, vector in zip(missing, encoded, strict=True):
73                 _vectors[(model_name, text)] = vector.tolist()
74     return [_vectors[(model_name, text)] for text in texts]

```

CODE app/ingest.py

```

1 # Internal and Confidential - Not for External Distribution.
2 """Ingest policy sections and their embeddings into the vector store."""
3
4 from pathlib import Path
5
6 from app.chunking import split
7 from app.db import DatabaseAdapter
8 from app.embeddings import embed_texts
9
10 POLICY_DIR = Path(__file__).resolve().parents[1] / "Policy"
11
12 _CREATE_TABLE = """
13 CREATE TABLE policy_chunks (
14     chunk_id TEXT PRIMARY KEY,
15     source_doc TEXT NOT NULL,
16     section TEXT NOT NULL,
17     section_title TEXT NOT NULL,
18     effective_date TEXT,
19     superseded_by TEXT,
20     parent_text TEXT NOT NULL,
21     text TEXT NOT NULL,
22     embedding vector(768) NOT NULL,
23     tsv tsvector GENERATED ALWAYS AS (to_tsvector('english', text)) STORED
24 )
25 """

```

```

26
27 _UPSERT = """
28 INSERT INTO policy_chunks (
29     chunk_id, source_doc, section, section_title, effective_date,
30     superseded_by, parent_text, text, embedding
31 )
32 VALUES (
33     %(chunk_id)s, %(source_doc)s, %(section)s, %(section_title)s,
34     %(effective_date)s, %(superseded_by)s, %(parent_text)s, %(text)s,
35     %(embedding)s
36 )
37 ON CONFLICT (chunk_id) DO UPDATE SET
38     source_doc = EXCLUDED.source_doc,
39     section = EXCLUDED.section,
40     section_title = EXCLUDED.section_title,
41     effective_date = EXCLUDED.effective_date,
42     superseded_by = EXCLUDED.superseded_by,
43     parent_text = EXCLUDED.parent_text,
44     text = EXCLUDED.text,
45     embedding = EXCLUDED.embedding
46 """
47
48
49 _CREATE_TSV_INDEX = """
50 CREATE INDEX IF NOT EXISTS policy_chunks_tsv_gin ON policy_chunks USING GIN (tsv)
51 """
52
53
54 def _ensure_schema(conn) -> None:
55     """Create the vector extension, table, and keyword index when missing.
56
57     A table from an older schema (no ``parent_text`` or ``tsv`` columns) is
58     dropped and rebuilt; the ingest that follows repopulates it.
59     """
60     conn.execute("CREATE EXTENSION IF NOT EXISTS vector")
61     columns = conn.execute(
62         """
63         SELECT column_name
64         FROM information_schema.columns
65         WHERE table_name = 'policy_chunks'
66             AND column_name IN ('parent_text', 'tsv')
67         """
68     ).fetchall()
69     if len(columns) < 2:
70         conn.execute("DROP TABLE IF EXISTS policy_chunks")
71         conn.execute(_CREATE_TABLE)
72         conn.execute(_CREATE_TSV_INDEX)
73
74
75 def _chunks() -> list[dict]:
76     chunks: list[dict] = []
77     for path in sorted(POLICY_DIR.glob("*.md")):
78         chunks.extend(split(path.read_text(encoding="utf-8"), path.name))
79     return chunks
80
81
82 def run(adapter: DatabaseAdapter) -> int:
83     """Replace stored policy chunks with every embedded policy document.
84
85     Args:
86         adapter: Provider of a managed vector-capable SQL connection.
87
88     Side Effects:
89         Reads every ``Policy/*.md`` file, creates the vector extension and
90         table when needed, upserts current chunks, deletes stale chunks,
91         and returns the number of chunks ingested.
92     """
93     chunks = _chunks()
94     vectors = embed_texts([chunk["text"] for chunk in chunks])
95     with adapter.connect() as conn:
96         _ensure_schema(conn)
97         ids = []
98         for chunk, vector in zip(chunks, vectors, strict=True):
99             conn.execute(_UPSERT, {**chunk, "embedding": vector})
100             ids.append(chunk["chunk_id"])
101         conn.execute(
102             "DELETE FROM policy_chunks WHERE chunk_id <> ALL(%s)",
103             (ids,),

```

```
104     )  
105     return len(chunks)
```

Deliverable 4: Basic RAG pipeline end to end

LIVE python -m app ask "What approvals do I need for a \$1,200 expense?"

```
{
  "answer": "For a $1,200 expense, you need Squad Leader + Dr. Nefario approval. Additionally, a receipt is required, and the expense must be reasonable, necessary, and documented with a written description specifying the evil purpose.",
  "citation": {
    "source_doc": "minion_expense_policy_2024.md",
    "effective_date": "January 15, 2024",
    "section": "minion_expense_policy_2024.md 4. Approval Thresholds",
    "retrieved_chunks": [
      {
        "source_doc": "minion_expense_policy_2024.md",
        "effective_date": "January 15, 2024",
        "section": "minion_expense_policy_2024.md 4. Approval Thresholds",
        "text": "- **Under $500:** Minion Squad Leader approval only.\n- **$500-$2,000:** Squad Leader + Dr. Nefario approval.\n- **Over $2,000:** Gru approval required, in person, said with full dramatic villain monologue energy whether you want it or not.\n- **Anything involving unauthorized weapon prototypes:** Triggers **Villain Protocol** immediately, no exceptions, this is how we lost the second sub-basement.",
        "distance": 0.3379465609951344
      },
      {
        "source_doc": "minion_expense_policy_2024.md",
        "effective_date": "January 15, 2024",
        "section": "minion_expense_policy_2024.md 3. Submission and Deadlines",
        "text": "Expense reports submitted within **60 days**. Late submission requires written approval from Dr. Nefario, who will look at you over his glasses for a long, long time before signing.",
        "distance": 0.3330400573269663
      },
      {
        "source_doc": "minion_expense_policy_2024.md",
        "effective_date": "January 15, 2024",
        "section": "minion_expense_policy_2024.md 2. General Principles",
        "text": "All expenses must be reasonable, necessary, and have receipt. Expenses under $25 (or 25 bananas, exchange rate fluctuates) need only written description. Description \"for evil purposes\" is not specific enough. We need to know WHICH evil purpose.",
        "distance": 0.3221657605991495
      },
      {
        "source_doc": "minion_expense_policy_2024.md",
        "effective_date": "January 15, 2024",
        "section": "minion_expense_policy_2024.md 5. Travel Expenses",
        "text": "- **Mileage (Minion Pod / Van):** Reimbursed at **$0.67 per mile**, tracked via the van's odometer, which Stuart insists is \"possessed\" and not merely \"driven badly by Stuart.\"\n- **Airfare:** Economy under 6 hours; premium economy for longer flights, pre-approved, and NO, \"the rocket\" does not count as economy, that is a whole different budget line and you know it.\n- **Lodging:** Reasonable rates; itemized folio required, including the $30 in-room banana snacks we already know you bought.\n- **Meals while traveling:** Per diem **$75/day** domestic, **$95/day** international, tips included. No reimbursement for \"emotional support tacos,\" even during a difficult mission.\n- **Corporate card:** Required for missions with anticipated expenses over $1,000. Personal card usage on major missions is a fast track to Villain Protocol.",
        "distance": 0.36245943871680153
      },
      {
        "source_doc": "minion_expense_policy_2024.md",
        "effective_date": "January 15, 2024",
        "section": "minion_expense_policy_2024.md 8. Client / Rival Villain Entertainment",
        "text": "Meetings with rival villains, meant to gather intel, reimbursable up to $150/person, with names and organizations documented, unless meeting devolves into \"who has the better lair,\" in which case that portion is on you personally.",
        "distance": 0.40623993793180924
      }
    ]
  }
}
```

Deliverable 5: Hybrid retrieval code and where it beats vector-only

CODE app/retrieve.py

```
1 # Internal and Confidential - Not for External Distribution.
2 """Hybrid policy retrieval: vector and keyword lanes fused by reciprocal rank."""
3
4 from typing import Literal
5
6 from app.db import DatabaseAdapter
7 from app.embeddings import embed_texts
8
9 Mode = Literal["hybrid", "vector"]
10
11 _VECTOR_SQL = """
12 SELECT chunk_id, source_doc, section, section_title, effective_date,
13        parent_text, embedding <=> %s::vector AS distance
14 FROM policy_chunks
15 WHERE superseded_by IS NULL
16 ORDER BY distance
17 LIMIT 20
18 """
19
20 _KEYWORD_SQL = """
21 SELECT chunk_id, source_doc, section, section_title, effective_date,
22        parent_text, embedding <=> %s::vector AS distance
23 FROM policy_chunks
24 WHERE superseded_by IS NULL AND tsv @@ websearch_to_tsquery('english', %s)
25 ORDER BY ts_rank(tsv, websearch_to_tsquery('english', %s)) DESC
26 LIMIT 20
27 """
28
29 _VECTOR_SQL_ALL = """
30 SELECT chunk_id, source_doc, section, section_title, effective_date,
31        parent_text, embedding <=> %s::vector AS distance
32 FROM policy_chunks
33 ORDER BY distance
34 LIMIT 20
35 """
36
37 _KEYWORD_SQL_ALL = """
38 SELECT chunk_id, source_doc, section, section_title, effective_date,
39        parent_text, embedding <=> %s::vector AS distance
40 FROM policy_chunks
41 WHERE tsv @@ websearch_to_tsquery('english', %s)
42 ORDER BY ts_rank(tsv, websearch_to_tsquery('english', %s)) DESC
43 LIMIT 20
44 """
45
46
47 def _candidate(row) -> dict:
48     return {
49         "chunk_id": row[0],
50         "source_doc": row[1],
51         "section": row[2],
52         "section_title": row[3],
53         "effective_date": row[4],
54         "text": row[5],
55         "distance": float(row[6]),
56     }
57
58
59 def fuse(vector_rows: list[dict], keyword_rows: list[dict], k: int = 60) -> list[tuple]:
60     ranks: dict[str, dict] = {}
61     for lane, rows in (("vector_rank", vector_rows), ("keyword_rank", keyword_rows)):
62         for rank, row in enumerate(rows, start=1):
63             ranks.setdefault(row["chunk_id"], {})[lane] = rank
64     scored = [
65         (
66             chunk_id,
67             1.0 / (k + lane.get("vector_rank", float("inf")))
68             + 1.0 / (k + lane.get("keyword_rank", float("inf"))),
69             lane.get("vector_rank", float("inf")),
70             lane.get("keyword_rank", float("inf")),
71         )
72         for chunk_id, lane in ranks.items()
```

```

73     ]
74     scored.sort(key=lambda entry: (-entry[1], entry[2], entry[3]))
75     return [(chunk_id, score) for chunk_id, score, _, _ in scored]
76
77
78 def search(
79     question: str,
80     adapter: DatabaseAdapter,
81     mode: Mode = "hybrid",
82     include_superseded: bool = False,
83 ) -> list[dict]:
84     """Fuse the vector and keyword lanes for an embedded employee question.
85
86     Args:
87         question: Employee question to embed and match against stored chunks.
88         adapter: Provider of a managed vector-capable SQL connection.
89         mode: ``hybrid`` runs both lanes; ``vector`` runs cosine search only.
90         include_superseded: Keep chunks whose document is marked superseded.
91         The lineage filter stays on by default so stale policy versions
92         never reach the prompt.
93
94     Returns:
95         Up to twenty chunk dictionaries ordered by fused reciprocal rank.
96     """
97     query_vector = embed_texts([question])[0]
98     vector_sql = _VECTOR_SQL_ALL if include_superseded else _VECTOR_SQL
99     keyword_sql = _KEYWORD_SQL_ALL if include_superseded else _KEYWORD_SQL
100    with adapter.connect() as conn:
101        vector_rows = [
102            _candidate(row) for row in conn.execute(vector_sql, (query_vector,)).fetchall()
103        ]
104        keyword_rows = (
105            []
106            if mode == "vector"
107            else [
108                _candidate(row)
109                for row in conn.execute(keyword_sql, (query_vector, question, question)).fetchall()
110            ]
111        )
112    by_chunk_id = {row["chunk_id"]: row for row in [*vector_rows, *keyword_rows]}
113    fused = fuse(vector_rows, keyword_rows)[:20]
114    return [
115        {**by_chunk_id[chunk_id], "rrf_score": float(score)}
116        for chunk_id, score in fused
117    ]

```

The A/B case is the golden form-zx-4491 ("What does form ZX-4491 authorize?", dress code s11). Its chunk sits at vector rank 26, outside the top-20 vector lane, but the keyword lane matches it exactly, so hybrid retrieves it and vector-only misses it.

Live A/B, same build

golden	hybrid	vector-only	
per-diem	PASS	PASS	
mileage-rate	PASS	PASS	
submission-window	PASS	PASS	
mid-approval-tier	PASS	PASS	
villain-protocol	PASS	PASS	
form-zx-4491	PASS	MISS	hybrid wins
per-diem-poisoned	PASS	PASS	
home-office-stipend	PASS	PASS	
bananaphone	PASS	PASS	
top-approval-tier	PASS	PASS	

hybrid summary: 10/10; vector-only summary: 9/10 (exit 1, nonzero because a golden MISSED).

Deliverable 6: Reranking code

Cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2 scores each (question, parent text) pair, keeps the best child per section, and passes the top five, best first, to generation.

CODE app/reranker.py

```
1 from typing import Protocol
2
3 from app.embeddings import _quiet_model_loading
4
5 MODEL_NAME = "cross-encoder/ms-marco-MiniLM-L-6-v2"
6
7 _model = None
8
9
10 class Reranker(Protocol):
11     def rank(self, question: str, chunks: list[dict]) -> list[dict]: ...
12
13
14 class CrossEncoderReranker:
15     def rank(self, question: str, chunks: list[dict], top_n: int = 5) -> list[dict]:
16         if not chunks:
17             return []
18         scores = self._predict([(question, chunk["text"]) for chunk in chunks])
19         order = sorted(range(len(chunks)), key=lambda index: (-scores[index], index))
20         selected: list[dict] = []
21         seen: set[tuple[str, str]] = set()
22         for index in order:
23             chunk = chunks[index]
24             key = (chunk["source_doc"], chunk["section"])
25             if key in seen:
26                 continue
27             seen.add(key)
28             selected.append(chunk)
29             if len(selected) == top_n:
30                 break
31         return selected
32
33     def _predict(self, pairs: list[tuple[str, str]]) -> list[float]:
34         global _model
35         if _model is None:
36             from sentence_transformers import CrossEncoder
37
38             with _quiet_model_loading():
39                 _model = CrossEncoder(MODEL_NAME)
40         return [float(score) for score in _model.predict(pairs)]
```

Deliverable 7: Evaluation test set and harness code

Ten golden cases (recall@5 by section prefix, case-insensitive fact checks, exact-refusal pin) and the harness that runs them through the real pipeline, cache bypassed.

CODE eval/goldens.json

```
1  [
2    {
3      "id": "per-diem",
4      "question": "How much can I spend on food each day?",
5      "expected": ["minion_expense_policy_2024:s5"],
6      "must_contain": ["$75", "domestic"],
7      "must_not_contain": ["$60", "$65"],
8      "kind": "answer"
9    },
10   {
11     "id": "mileage-rate",
12     "question": "What is the mileage reimbursement rate for the Minion Van?",
13     "expected": ["minion_expense_policy_2024:s5"],
14     "must_contain": ["$0.67"],
15     "must_not_contain": ["$0.56"],
16     "kind": "answer"
17   },
18   {
19     "id": "submission-window",
20     "question": "How long do I have to submit my expense report?",
21     "expected": ["minion_expense_policy_2024:s3"],
22     "must_contain": ["60 days"],
23     "must_not_contain": ["90 days"],
24     "kind": "answer"
25   },
26   {
27     "id": "mid-approval-tier",
28     "question": "What approvals do I need for a $1,200 expense?",
29     "expected": ["minion_expense_policy_2024:s4"],
30     "must_contain": ["Squad Leader", "Nefario"],
31     "must_not_contain": [],
32     "kind": "answer"
33   },
34   {
35     "id": "villain-protocol",
36     "question": "What triggers the Villain Protocol?",
37     "expected": ["minion_travel_security_policy:s6"],
38     "must_contain": ["unauthorized weapon prototype"],
39     "must_not_contain": [],
40     "kind": "hybrid_demo"
41   },
42   {
43     "id": "form-zx-4491",
44     "question": "What does form ZX-4491 authorize?",
45     "expected": ["minion_dress_code_policy:s11"],
46     "must_contain": ["$40"],
47     "must_not_contain": [],
48     "kind": "hybrid_demo"
49   },
50   {
51     "id": "per-diem-poisoned",
52     "question": "What is the daily meal per diem for domestic travel?",
53     "expected": ["minion_expense_policy_2024:s5"],
54     "must_contain": ["$75"],
55     "must_not_contain": ["$60", "$65", "$80"],
56     "kind": "poisoned"
57   },
58   {
59     "id": "home-office-stipend",
60     "question": "Is there a stipend for my home office setup?",
61     "expected": ["minion_expense_policy_2024:s6"],
62     "must_contain": ["$500"],
63     "must_not_contain": [],
64     "kind": "answer"
65   },
66   {
67     "id": "bananaphone",
68     "question": "Does the policy cover bananaphone accessories?",
```

```

69     "expected": [],
70     "must_contain": ["The provided policy does not answer this question."],
71     "must_not_contain": [],
72     "kind": "refusal"
73 },
74 {
75     "id": "top-approval-tier",
76     "question": "Who has to approve expenses over $2,000?",
77     "expected": ["minion_expense_policy_2024:s4"],
78     "must_contain": ["Gru"],
79     "must_not_contain": [],
80     "kind": "answer"
81 }
82 ]

```

CODE app/evaluate.py

```

1  """The fixed exam: golden cases, recall and answer checks, the harness run."""
2
3  import json
4  from pathlib import Path
5  from typing import Literal
6
7  from pydantic import BaseModel
8
9  from app.db import DatabaseAdapter
10 from app.generate import LanguageModel, generate
11 from app.reranker import Reranker
12 from app.retrieve import Mode, search
13 from app.schemas import AskResponse
14 from app.validate import REFUSAL
15
16 GOLDENS_PATH = Path(__file__).resolve().parents[1] / "eval" / "goldens.json"
17
18 Kind = Literal["answer", "hybrid_demo", "poisoned", "refusal"]
19
20
21 class Golden(BaseModel):
22     id: str
23     question: str
24     expected: list[str]
25     must_contain: list[str]
26     must_not_contain: list[str]
27     kind: Kind
28
29
30 def load_goldens(path: Path = GOLDENS_PATH) -> list[Golden]:
31     return [Golden.model_validate(case) for case in json.loads(path.read_text(encoding="utf-8"))]
32
33
34 def section_prefix_hit(chunk_id: str, expected_sections: list[str]) -> bool:
35     return any(
36         chunk_id.startswith(f"{expected_section}:")
37         for expected_section in expected_sections
38     )
39
40
41 def recall_pass(ranked_chunk_ids: list[str], golden: Golden) -> bool:
42     if not golden.expected:
43         return True
44     return any(section_prefix_hit(chunk_id, golden.expected) for chunk_id in ranked_chunk_ids)
45
46
47 def answer_failures(response: AskResponse, golden: Golden) -> list[str]:
48     if golden.kind == "refusal":
49         if response.answer != REFUSAL:
50             return [f"expected the exact refusal, got: {response.answer!r}"]
51         if response.citation is not None:
52             return ["refusal must carry no citation"]
53         return []
54     lowered = response.answer.lower()
55     failures = [
56         f"missing required fact {fact!r}"
57         for fact in golden.must_contain
58         if fact.lower() not in lowered
59     ]
60     failures += [
61         f"forbidden fact present {fact!r}"

```

```

62         for fact in golden.must_not_contain
63             if fact.lower() in lowered
64         ]
65     return failures
66
67
68 def run_exam(
69     goldens: list[Golden],
70     *,
71     adapter: DatabaseAdapter,
72     model: LanguageModel,
73     reranker: Reranker,
74     mode: Mode = "hybrid",
75     include_superseded: bool = False,
76 ) -> dict:
77     """Each golden through the exact ask path minus the question cache."""
78     results = []
79     for golden in goldens:
80         candidates = search(
81             golden.question,
82             adapter,
83             mode=mode,
84             include_superseded=include_superseded,
85         )
86         ranked = reranker.rank(golden.question, candidates)
87         response = generate(golden.question, ranked, model)
88         ranked_chunk_ids = [chunk["chunk_id"] for chunk in ranked]
89         recall = recall_pass(ranked_chunk_ids, golden)
90         failures = answer_failures(response, golden)
91         results.append(
92             {
93                 "id": golden.id,
94                 "kind": golden.kind,
95                 "question": golden.question,
96                 "recall": recall,
97                 "answer_failures": failures,
98                 "passed": recall and not failures,
99                 "answer": response.answer,
100                 "citation": (
101                     response.citation.model_dump() if response.citation else None
102                 ),
103                 "ranked_chunk_ids": ranked_chunk_ids,
104             }
105         )
106     return {
107         "retriever": mode,
108         "include_superseded": include_superseded,
109         "results": results,
110         "summary": {
111             "total": len(results),
112             "passed": sum(result["passed"] for result in results),
113             "recall_hits": sum(result["recall"] for result in results),
114             "answer_passes": sum(
115                 not result["answer_failures"] for result in results
116             ),
117         },
118     }
119
120
121 def format_table(record: dict) -> str:
122     summary = record["summary"]
123     lines = [
124         f"retriever={record['retriever']} goldens={summary['total']} "
125         f"passed={summary['passed']} recall={summary['recall_hits']}/{summary['total']} "
126         f"answers={summary['answer_passes']}/{summary['total']}",
127         "",
128         f"{'id':<22}{'kind':<13}{'recall':<8}{'answer':<8}result",
129     ]
130     for result in record["results"]:
131         lines.append(
132             f"{'id':<22}{'kind':<13}"
133             f"{'hit' if result['recall'] else 'MISS':<8}"
134             f"{'ok' if not result['answer_failures'] else 'FAIL':<8}"
135             f"{'PASS' if result['passed'] else 'FAIL'}"
136         )
137         lines.extend(f"    {failure}" for failure in result["answer_failures"])
138     return "\n".join(lines)

```

Deliverable 8: Evaluation harness terminal output with recall and accuracy

LIVE python -m app eval

```
retriever=hybrid goldens=10 passed=10 recall=10/10 answers=10/10

id          kind      recall answer result
per-diem    answer    hit    ok    PASS
mileage-rate answer    hit    ok    PASS
submission-window answer    hit    ok    PASS
mid-approval-tier answer    hit    ok    PASS
villain-protocol hybrid_demo hit    ok    PASS
form-zx-4491 hybrid_demo hit    ok    PASS
per-diem-poisoned poisoned hit    ok    PASS
home-office-stipend answer    hit    ok    PASS
bananaphone refusal    hit    ok    PASS
top-approval-tier answer    hit    ok    PASS
```

Deliverable 9: Planted-issue question, flawed answer, and written diagnosis

LIVE python -m app ask "How much can I spend on food each day?"

```
{"answer": "You can spend up to $75 per day on meals while traveling domestically and $95 per day internationally, with tips included. This is the per diem rate for meals while traveling.", "citation": {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 5. Travel Expenses"}, "retrieved_chunks": [{"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 5. Travel Expenses", "text": "- **Mileage (Minion Pod / Van):** Reimbursed at **$0.67 per mile**, tracked via the van's odometer, which Stuart insists is \"possessed\" and not merely \"driven badly by Stuart.\"\\n- **Airfare:** Economy under 6 hours; premium economy for longer flights, pre-approved, and NO, \"the rocket\" does not count as economy, that is a whole different budget line and you know it.\\n- **Lodging:** Reasonable rates; itemized folio required, including the $30 in-room banana snacks we already know you bought.\\n- **Meals while traveling:** Per diem **$75/day** domestic, **$95/day** international, tips included. No reimbursement for \"emotional support tacos,\" even during a difficult mission.\\n- **Corporate card:** Required for missions with anticipated expenses over $1,000. Personal card usage on major missions is a fast track to Villain Protocol.\""}, {"distance": 0.37131161850024086}, {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 2. General Principles", "text": "All expenses must be reasonable, necessary, and have receipt. Expenses under $25 (or 25 bananas, exchange rate fluctuates) need only written description. Description \"for evil purposes\" is not specific enough. We need to know WHICH evil purpose.\""}, {"distance": 0.37032704790929816}, {"source_doc": "minion_pto_policy.md", "effective_date": "January 1, 2024", "section": "minion_pto_policy.md 2. PTO Accrual", "text": "Full-time Minions accrue **1.67 days per month**, totaling **20 days per year**. Part-time Minions (currently: none, everyone here is very full-time, very committed, slightly too committed) accrue proportionally. Cannot use PTO in first 30 days, so no, you cannot request \"Day One Nap\" as vacation, Stuart, we discussed this.\""}, {"distance": 0.40931736198944835}, {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 8. Client / Rival Villain Entertainment", "text": "Meetings with rival villains, meant to gather intel, reimbursable up to $150/person, with names and organizations documented, unless meeting devolves into \"who has the better lair,\" in which case that portion is on you personally.\""}, {"distance": 0.43983113166001253}, {"source_doc": "minion_pto_policy.md", "effective_date": "January 1, 2024", "section": "minion_pto_policy.md 7. Bereavement Leave", "text": "**5 paid days** for immediate family, **2 paid days** for extended family or, per lair precedent, one (1) beloved banana that was \"like a brother, honestly.\" No Joke Protocol jokes during bereavement. This is the one rule nobody has ever broken. We remain, as an organization, proud of this.\""}, {"distance": 0.4063771666307794}]}
```

LIVE python -m app ask "How much can I spend on food each day?" --include-superseded

```
{"answer": "You can spend up to $75 per day on food while traveling domestically, and $95 per day on food while traveling internationally. Tips are included in these amounts.", "citation": {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 5. Travel Expenses"}, "retrieved_chunks": [{"source_doc": "minion_expense_policy_2021.md", "effective_date": "June 1, 2021", "section": "minion_expense_policy_2021.md 5. Travel Expenses", "text": "- **Mileage (Minion Pod / Van):** Reimbursed at **$0.56 per mile**.\\n- **Airfare:** Economy under 6 hours; premium economy for longer flights, pre-approved.\\n- **Lodging:** Reasonable rates; itemized folio required.\\n- **Meals while traveling:** Per diem **$60/day** domestic, **$80/day** international, tips included.\""}, {"distance": 0.3736047083433027}, {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 5. Travel Expenses", "text": "- **Mileage (Minion Pod / Van):** Reimbursed at **$0.67 per mile**, tracked via the van's odometer, which Stuart insists is \"possessed\" and not merely \"driven badly by Stuart.\"\\n- **Airfare:** Economy under 6 hours; premium economy for longer flights, pre-approved, and NO, \"the rocket\" does not count as economy, that is a whole different budget line and you know it.\\n- **Lodging:** Reasonable rates; itemized folio required, including the $30 in-room banana snacks we already know you bought.\\n- **Meals while traveling:** Per diem **$75/day** domestic, **$95/day** international, tips included. No reimbursement for \"emotional support tacos,\" even during a difficult mission.\\n- **Corporate card:** Required for missions with anticipated expenses over $1,000. Personal card usage on major missions is a fast track to Villain Protocol.\""}, {"distance": 0.37131161850024086}, {"source_doc": "minion_expense_policy_2024.md", "effective_date": "January 15, 2024", "section": "minion_expense_policy_2024.md 2. General Principles", "text": "All expenses must be reasonable, necessary, and have receipt. Expenses under $25 (or 25 bananas, exchange rate fluctuates) need only written description. Description \"for evil purposes\" is not specific enough. We need to know WHICH evil purpose.\""}, {"distance": 0.37032704790929816}, {"source_doc": "minion_pto_policy.md", "effective_date": "January 1, 2024", "section": "minion_pto_policy.md 2. PTO Accrual", "text": "Full-time Minions accrue **1.67 days per month**, totaling **20 days per year**. Part-time Minions (currently: none, everyone here is very full-time, very committed, slightly too committed) accrue proportionally. Cannot use PTO in first 30 days, so no, you cannot request \"Day One Nap\" as vacation, Stuart, we discussed this.\""}, {"distance": 0.40931736198944835}, {"source_doc": "minion_expense_policy_2021.md", "effective_date": "June 1, 2021", "section": "minion_expense_policy_2021.md 2. General Principles", "text": "All expenses must be reasonable, necessary, and have receipt. Expenses under $20 need only written description.\""}, {"distance": 0.4213132422186646}]}
```

The filtered ask returns \$75 citing 2024; the bypassed ask re-admits the stale 2021 duplicate so the \$60 answer can be reproduced on demand. Bypassed asks are never cached.

Written diagnosis (docs/part6-diagnosis.md)

Part 6: Diagnosing the Planted Data Quality Issue

The corpus ships with exactly one realistic defect: a stale 2021 duplicate of the expense policy filed under the same document ID as the current 2024 version (both are LAIR-POL-009), with conflicting numbers (the 2021 domestic meal per diem is \$60/day against the 2024 \$75/day, and the mileage rate is \$0.56 against \$0.67). This note documents the probe, the three-question diagnosis, the fix that was chosen, and how to reproduce the flawed behavior on demand.

The probe question

```
```\npython -m app ask "How much can I spend on food each day?"\n```
```

Before the lineage fix, this question could return \$60/day citing the 2021 duplicate. Today it returns \$75/day citing the 2024 section.

#### ## The three-question method

**\*\*1. Was it retrieval?\*\*** No. Retrieval is measured directly: the golden exam's per-diem case requires the 2024 travel-expenses section in the final five and passes (``python -m app eval``, recall 10/10). Both lanes and the reranker deliver the governing section; nothing in the retrieval path prefers the stale document.

**\*\*2. Was it generation or grounding?\*\*** No. The generator answers only from the excerpts it is given; the citation gate forces the exact refusal whenever the model names a section that was not retrieved, so it cannot invent a \$60 answer out of thin air. Given only current sections, the answer is correct every time.

**\*\*3. Was it the source data?\*\*** Yes. With the lineage filter bypassed (reproducing the pre-fix state of the corpus), the superseded 2021 section re-enters both retrieval lanes and reaches the model alongside the current one, and per-diem answers quoting \$60/day become possible again. The conflicting number exists only because a stale duplicate lives in the source set under one document ID. **\*\*Root cause: the source data, not the retrieval logic and not the generation logic.\*\***

#### ## The fix: lineage, not prompt patches

The fix stamps lineage at ingest and filters at retrieval. The 2021 file carries a ``**Superseded-By:** minion_expense_policy_2024.md`` header; chunking stores it in the ``superseded_by`` column; both retrieval lanes filter ``superseded_by IS NULL``, so normal answers never see superseded rules.

Explicitly rejected: patching the prompt to say "prefer 2024 documents" or "the per diem is \$75". Prompt patches hardcode facts that go stale the next time a policy changes, do not survive new documents, and mask data problems instead of fixing them. The poisoned golden (``per-diem-poisoned``) pins the fix permanently: it demands the current number and fails the exam if the stale one ever leaks back into an answer.

#### ## Reproducing the defect on demand

The ``--include-superseded`` flag on `ask` and `eval` is the opt-in lineage bypass. It is strictly opt-in, defaults preserve the filtered behavior, and bypassed asks never touch the question cache.

Deterministic retrieval-level proof (the stale section reaches the model's candidate set only under the flag):

```
```
```

```
pytest -q -k lineage
python -m app eval --include-superseded --output eval/record-bypass.json
'''
```

The record's `ranked\_chunk\_ids` then contain
`minion\_expense\_policy\_2021:s5` for the per-diem goldens; without the
flag no 2021 id appears anywhere in the record.

Flawed-answer capture with the real generator (owner's screenshot for
deliverable 9):

```
'''
python -m app ingest
python -m app ask "How much can I spend on food each day?"
python -m app ask "How much can I spend on food each day?" --include-superseded
'''
```

The first command shows the correct \$75 answer with the 2024 citation.
The second re-exposes the pre-fix behavior: the stale section is in the
model's excerpts, and the capture shows either the flawed \$60 answer
citing 2021 or the recovered behavior citing 2024, depending on which
excerpt the generator commits to at temperature 0. Either way the
diagnosis stands on the deterministic retrieval-level evidence above:
with bad source data admitted, the pipeline faithfully serves the bad
number; the defect is in the corpus, and the lineage filter is what
keeps it out.

Deliverable 10: Source attribution output

LIVE citation from the Deliverable 4 ask

```
{
  "source_doc": "minion_expense_policy_2024.md",
  "effective_date": "January 15, 2024",
  "section": "minion_expense_policy_2024.md 4. Approval Thresholds"
}
```

Every answer carries document, effective date, and section; the full retrieved chunk list rides along in the same response.

Deliverable 11: Passing pipeline run (CI)

PROVIDED

Passing pipeline run (CI)

Summary

All jobs

fast

full

Run details

Usage

Workflow file

fast

succeeded 23 minutes ago in 8m 14s

Search logs

Refresh

Settings

> Run actions/setup-python@v517s

> Restore the embedding and reranking models3s

> Install dependencies1m 11s

Download models when the cache missed0s

Save the embedding and reranking models0s

> Bootstrap the store before the tests2m 18s

> Fast suite (real retrieval and reranking, faked generation)4m 0s

1 Run pytest -q

12 [79%]

13 [100%]

14 ===== tests coverage =====

15 coverage: platform linux, python 3.12.14-final-0

16

17 Name Stmts Miss Cover Missing

18

19 app/__init__.py 0 0 100%

20 app/__main__.py 3 3 0% 4-7

21 app/cache.py 25 0 100%

22 app/chunking.py 69 4 94% 39, 52, 68, 77

23 app/cli.py 61 5 92% 68-77, 89, 122

24 app/config.py 9 0 100%

25 app/db.py 4 0 100%

26 app/embeddings.py 39 0 100%

Latest green run: 36196572643 (2026-09-25T22:24:45)
<https://github.com/kenmann01/minirag/actions/runs/36196572643>

Recent runs (gh run list --workflow ci)

| id | title | conclusion | created |
|-------------|--|------------|---------------------|
| 36196572643 | chore: add submission PDF builder script | success | 2026-09-25T22:24:45 |
| 36181246748 | chore: added vector query comparison and other eval gates | success | 2026-09-25T19:41:47 |
| 36166033476 | chore: added html view page to vizually show the trace | success | 2026-09-25T17:16:01 |
| 36152801357 | fix: fixed mermaid render issues and added citations for answers | success | 2026-09-25T15:14:13 |
| 36149578570 | chore: Trace a cache miss | success | 2026-09-25T14:45:09 |

LIVE

gh run view 36196572643

✓ feat/rag ci · 36196572643

Triggered via push about 23 minutes ago

JOBS

✓ fast in 8m14s (ID 108273660612)

- full (ID 108273661608)

ANNOTATIONS

! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/cache/restore@v4, actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

fast: .github#2

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>"

fast: .github#1

For more information about a job, try: gh run view --job=<job-id>

View this run on GitHub: <https://github.com/kenmann01/minirag/actions/runs/36196572643>

Screenshot the green run page from this URL for the graded image, or drop it into submission_assets/11-ci.png and rebuild.

Appendix: pipeline trace (ask-trace viewer)

Beyond the rubric: `python -m app serve` opens **app/static/index.html** on `http://127.0.0.1:8765`. Each ask returns every retrieval stage: stored chunks, cache hit/miss, the vector lane, the parsed keyword query and its lane, RRF fusion, cross-encoder ranking, what passed to generation, and the cache-store decision.

Live trace for *What triggers the Villain Protocol?* (canned generation, real retrieval stages, cache bypassed neither: this went through the real path):

| stage | detail |
|-----------------------|------------|
| ingest | 101 chunks |
| cache | miss |
| question_embedding | 20 chunks |
| keyword_search | 4 chunks |
| rrf | 20 chunks |
| cross_encoder | 5 chunks |
| passed_for_generation | 5 chunks |
| store | False |

Full stage-by-stage chunk lists are in `submission/trace-villain-protocol.json`; the HTML viewer renders these interactively. Screenshot it for extra credit.

The trace recorder (`app/trace.py`) and its server (`app/serve.py`)

CODE `app/trace.py`

```
1 # Internal and Confidential - Not for External Distribution.
2 """Run one ask and record what each retrieval stage held."""
3
4 from app.cache import lookup, store
5 from app.db import DatabaseAdapter
6 from app.embeddings import embed_texts
7 from app.generate import REFUSAL, LanguageModel, generate
8 from app.reranker import Reranker
9 from app.retrieve import (
10     _KEYWORD_SQL,
11     _KEYWORD_SQL_ALL,
12     _VECTOR_SQL,
13     _VECTOR_SQL_ALL,
14     _candidate,
15     fuse,
16 )
17
18 _INGEST_SQL = """
19 SELECT chunk_id, source_doc, section, section_title, effective_date, superseded_by
20 FROM policy_chunks
21 ORDER BY source_doc, section, chunk_id
22 """
23
24
25 def trace_ask(
26     question: str,
27     *,
28     database_adapter: DatabaseAdapter,
29     language_model: LanguageModel,
30     reranker: Reranker,
31     include_superseded: bool = False,
32 ) -> dict:
33     """Answer a question and return the answer JSON plus each stage of that run."""
34     if not question.strip():
35         return {"error": "Enter a question."}
36     try:
37         return _trace(
38             question,
39             database_adapter=database_adapter,
40             language_model=language_model,
41             reranker=reranker,
42             include_superseded=include_superseded,
```

```

43     )
44     except Exception as exc:
45         return {"error": str(exc)}
46
47
48 def _trace(
49     question: str,
50     *,
51     database_adapter: DatabaseAdapter,
52     language_model: LanguageModel,
53     reranker: Reranker,
54     include_superseded: bool,
55 ) -> dict:
56     if not include_superseded:
57         cached = lookup(question, database_adapter)
58         if cached is not None:
59             return {
60                 "response": cached.model_dump(),
61                 "trace": [
62                     {"name": "cache", "status": "hit", "response": cached.model_dump()},
63                 ],
64             }
65     stored_chunks = _stored_chunks(database_adapter)
66     vector_rows, keyword_rows, keyword_query = _lanes(
67         question, database_adapter, include_superseded=include_superseded
68     )
69     by_chunk_id = {row["chunk_id"]: row for row in [*vector_rows, *keyword_rows]}
70     fused = [
71         {**by_chunk_id[chunk_id], "rrf_score": float(score)}
72         for chunk_id, score in fuse(vector_rows, keyword_rows)[:20]
73     ]
74     ranked = reranker.rank(question, fused)
75     response = generate(question, ranked, language_model)
76     stored = response.answer != REFUSAL and not include_superseded
77     if stored:
78         store(question, response, database_adapter)
79     trace = [{"name": "ingest", "chunks": stored_chunks}]
80     if not include_superseded:
81         trace.append({"name": "cache", "status": "miss"})
82     trace.extend(
83         [
84             {"name": "question_embedding", "chunks": vector_rows},
85             {"name": "keyword_search", "query": keyword_query, "chunks": keyword_rows},
86             {"name": "rrf", "chunks": fused},
87             {"name": "cross_encoder", "chunks": ranked},
88             {
89                 "name": "passed_for_generation",
90                 "chunks": ranked,
91                 "answer": response.answer,
92                 "citation": response.citation.model_dump() if response.citation else None,
93             },
94             {"name": "store", "stored": stored},
95         ]
96     )
97     return {"response": response.model_dump(), "trace": trace}
98
99
100 def _stored_chunks(adapter: DatabaseAdapter) -> list[dict]:
101     with adapter.connect() as conn:
102         rows = conn.execute(_INGEST_SQL).fetchall()
103     return [
104         {
105             "chunk_id": row[0],
106             "source_doc": row[1],
107             "section": row[2],
108             "section_title": row[3],
109             "effective_date": row[4],
110             "superseded": row[5] is not None,
111         }
112         for row in rows
113     ]
114
115
116 def _lanes(
117     question: str, adapter: DatabaseAdapter, *, include_superseded: bool
118 ) -> tuple[list[dict], list[dict], str]:
119     query_vector = embed_texts([question])[0]
120     vector_sql = _VECTOR_SQL_ALL if include_superseded else _VECTOR_SQL

```

```

121 keyword_sql = _KEYWORD_SQL_ALL if include_superseded else _KEYWORD_SQL
122 with adapter.connect() as conn:
123     vector_rows = [
124         _candidate(row) for row in conn.execute(vector_sql, (query_vector,)).fetchall()
125     ]
126     keyword_query = conn.execute(
127         "SELECT websearch_to_tsquery('english', %s)::text",
128         (question,),
129     ).fetchone()[0]
130     keyword_rows = [
131         _candidate(row)
132         for row in conn.execute(keyword_sql, (query_vector, question, question)).fetchall()
133     ]
134     return vector_rows, keyword_rows, keyword_query or ""

```

CODE app/serve.py

```

1 # Internal and Confidential - Not for External Distribution.
2 """Serve the ask-trace page on localhost."""
3
4 import json
5 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
6 from pathlib import Path
7
8 from app.db import DatabaseAdapter
9 from app.generate import LanguageModel
10 from app.pgadapter import PgAdapter
11 from app.reranker import CrossEncoderReranker, Reranker
12 from app.trace import trace_ask
13
14 _PAGE = Path(__file__).resolve().parent / "static" / "index.html"
15
16
17 def serve(
18     host: str = "127.0.0.1",
19     port: int = 8765,
20     *,
21     database_adapter: DatabaseAdapter | None = None,
22     language_model: LanguageModel | None = None,
23     reranker: Reranker | None = None,
24 ) -> None:
25     """Serve the trace page until the process is stopped."""
26     adapter = database_adapter or PgAdapter()
27     if language_model is None:
28         from app.ollama import OllamaAdapter
29
30         language_model = OllamaAdapter()
31     ranker = reranker or CrossEncoderReranker()
32     handler = _handler(adapter, language_model, ranker)
33     server = ThreadingHTTPServer((host, port), handler)
34     print(f"ask trace at http://{host}:{port}", flush=True)
35     server.serve_forever()
36
37
38 def _handler(
39     adapter: DatabaseAdapter,
40     language_model: LanguageModel,
41     reranker: Reranker,
42 ) -> type[BaseHTTPRequestHandler]:
43     class Handler(BaseHTTPRequestHandler):
44         def do_GET(self) -> None:
45             if self.path.split("?", 1)[0] != "/":
46                 self.send_error(404)
47                 return
48             body = _PAGE.read_bytes()
49             self.send_response(200)
50             self.send_header("Content-Type", "text/html; charset=utf-8")
51             self.send_header("Content-Length", str(len(body)))
52             self.end_headers()
53             self.wfile.write(body)
54
55         def do_POST(self) -> None:
56             if self.path.split("?", 1)[0] != "/ask":
57                 self.send_error(404)
58                 return
59             length = int(self.headers.get("Content-Length", "0"))
60             payload = json.loads(self.rfile.read(length) or b"{}")
61             result = trace_ask(

```

```
62         str(payload.get("question", "")),
63         database_adapter=adapter,
64         language_model=language_model,
65         reranker=reranker,
66         include_superseded=bool(payload.get("include_superseded", False)),
67     )
68     body = json.dumps(result).encode()
69     self.send_response(200)
70     self.send_header("Content-Type", "application/json")
71     self.send_header("Content-Length", str(len(body)))
72     self.end_headers()
73     self.wfile.write(body)
74
75     def log_message(self, format: str, *args) -> None:
76         return
77
78     return Handler
```